	Pimpri Chinchwad Education Trust's **Pimpri Chinchwad college of Engineering**
Assignment No. 7	
Course:- Database Management System Laboratory(BIT24PC04)	
Name:- Mahadev Suresh Jundale	
PRN:- 125B2F007	

PROBLEM STATEMENT:-

No SQL: Design and Develop Queries using CRUD operations. (Use CRUD operations, SAVE method, logical operators, joins etc.).

THEORY:-

CASE:- ECOMMERCE AND ORDER MANAGEMENT SYSTEM

1. INTRODUCTION TO NOSQL

NoSQL stands for "Not Only SQL" and refers to non-relational databases designed to handle large volumes of structured, semi-structured, and unstructured data. Unlike traditional SQL databases, NoSQL databases do not use fixed table schemas and allow flexible data storage using models such as documents, key-value pairs, columns, and graphs.

NoSQL databases are widely used in modern applications like e-commerce, social media, and real-time systems where scalability and performance are important. They support horizontal scaling, high availability, and distributed architecture, making them suitable for handling large data and high user traffic.

In an E-Commerce and Order Management System, NoSQL can store product details, customer data, and orders with flexible structure, allowing easy updates and efficient data management.

2. FEATURES OF NOSQL

NoSQL databases provide several features that make them suitable for modern applications and large-scale systems.

- **Schema-less Database**

NoSQL databases do not require a fixed schema. Data can be stored in different formats, and structure can be changed easily without modifying the entire database.

- **High Scalability**

NoSQL databases support horizontal scaling, which means data can be distributed across multiple servers. This improves performance and handles large amounts of data efficiently.

- **Flexible Data Model**

NoSQL supports multiple data models such as document, key-value, column, and graph. This flexibility allows storing structured, semi-structured, and unstructured data.

- **High Performance**

NoSQL databases are optimized for fast data access and retrieval. They provide better performance for large datasets and real-time applications.

- **Distributed Architecture**

NoSQL databases run on distributed systems where data is stored across multiple machines. This ensures reliability and fault tolerance.

- **Handles Big Data**

NoSQL databases are designed to handle huge volumes of data generated by modern applications such as e-commerce, social media, and analytics systems.

3. WHAT IS MONGODB

MongoDB is a NoSQL document-oriented database that stores data in the form of JSON-like documents. Instead of tables and rows used in relational databases, MongoDB uses collections and documents to store data. Each document contains key-value pairs and can have a flexible structure, allowing different documents in the same collection to have different fields.

MongoDB is designed for high performance, scalability, and flexibility. It supports large volumes of data and allows easy modification of data without changing the entire database structure. MongoDB is widely used in modern applications such as e-commerce websites, content management systems, and real-time applications.

MongoDB Basic Structure

- Database – Contains collections
- Collection – Group of documents
- Document – Set of key-value pairs
- Field – Individual data element inside document

4. FEATURES OF MONGODB

MongoDB provides several features that make it suitable for modern applications and large datasets.

- **Schema-less Structure**

MongoDB does not require a fixed schema. Documents in the same collection can have different fields, making data storage flexible.

- **Document-Oriented Storage**

MongoDB stores data in JSON-like documents. This allows storing complex and hierarchical data in a single document.

- **High Performance**

MongoDB provides fast read and write operations. It is optimized for handling large amounts of data efficiently.

- **Indexing Support**

MongoDB supports indexing to improve query performance. Indexes help in faster searching and sorting of data.

- **Replication**

MongoDB supports replication using replica sets. Data is copied across multiple servers to ensure availability and reliability.

5. Difference Between SQL and NoSQL

SQL and NoSQL databases differ in structure, scalability, and data handling. SQL databases use tables with fixed schemas, while NoSQL databases use flexible data models.

SQL Database	NoSQL Database
Uses relational model	Uses non-relational model
Data stored in tables	Data stored in collections/documents
Fixed schema	Dynamic schema
Uses rows and columns	Uses JSON-like documents
Supports joins	Limited joins or embedded data
Vertical scaling	Horizontal scaling
Structured data only	Structured and unstructured data
Examples: MySQL, Oracle	Examples: MongoDB, Cassandra

SQL Database

SQL databases store data in tables with predefined structure. They use relationships and joins to connect multiple tables. These databases are suitable for structured data and transactional systems.

NoSQL Database

NoSQL databases store data in flexible formats such as documents and key-value pairs. They are suitable for large-scale applications, big data, and systems requiring high performance and scalability.

6. CRUD OPERATIONS IN MONGODB

CRUD operations refer to Create, Read, Update, and Delete operations used to manage data in MongoDB collections.

Create Operation

Used to insert new documents into a collection.

Syntax:

```
db.collection_name.insertOne(document)
db.collection_name.insertMany([document1, document2])
```

- insertOne() – Insert single document
- insertMany() – Insert multiple documents
- Adds new records to collection

Read Operation

Used to retrieve documents from a collection.

Syntax:

```
db.collection_name.find()
db.collection_name.find(condition)
db.collection_name.findOne(condition)
```

- find() – Retrieve multiple documents
- findOne() – Retrieve single document
- Used with conditions and filters

Update Operation

Used to modify existing documents.

Syntax:

```
db.collection_name.updateOne(condition, {$set: {field:value}})
db.collection_name.updateMany(condition, {$set: {field:value}})
```

- updateOne() – Update single document
- updateMany() – Update multiple documents
- $set operator used for updating values

Delete Operation

Used to remove documents from a collection.

Syntax:

db.collection_name.deleteOne(condition)
db.collection_name.deleteMany(condition)

- deleteOne() – Delete single document
- deleteMany() – Delete multiple documents
- Removes unwanted data from collection

Save Method in nosql

The save() method is used to insert a new document or update an existing document in a collection. If the document already exists, it updates it; otherwise, it inserts a new document.

Syntax
db.collection_name.save(document)
Example (Insert)
db.customers.save({
customer_id: 1,
name: "Rahul",
city: "Pune"
})

6.OPERATORS IN NOSQL (MONGODB)

Operators in MongoDB are used to filter and retrieve documents based on conditions.

1. Comparison Operators

Used to compare values.

Syntax:

db.collection_name.find({field: {operator: value}})

Examples

- **Equal to**

db.products.find({price: {$eq: 50000}})

- **Greater than**

db.products.find({price: {$gt: 20000}})

- **Less than**

db.products.find({price: {$lt: 60000}})

- **Not equal**

db.products.find({price: {$ne: 30000}})

<u>Operators:</u>

- $eq Equal
- $ne Not equal
- $gt Greater than
- $lt Less than
- $gte Greater than equal
- $lte Less than equal

2. Logical Operators

Used to apply multiple conditions.

- **AND Operator**

```
db.products.find({
$and:[
{price: {$gt: 20000}},
{stock: {$gt: 5}}
]
})
```

- **OR Operator**

```
db.customers.find({
$or:[
{city: "Pune"},
{city: "Mumbai"}
]
})
```

- **NOT Operator**

```
db.products.find({
price: {$not: {$gt: 50000}}
})
```

3. Element Operators

Used to check field existence.

db.customers.find({email: {$exists: true}})

4. In Operator

```
db.products.find({
city: {$in: ["Pune","Mumbai"]}
})
```

5. Not In Operator

```
db.products.find({
city: {$nin: ["Pune","Delhi"]}
})
```

7.JOINS IN NOSQL (MONGODB)

MongoDB does not support joins like relational SQL databases. Instead, MongoDB uses embedding, referencing, and the $lookup operator to combine data from multiple collections. Joins in MongoDB are mainly performed using the aggregation framework.

The $lookup operator is used to perform join-like operations between two collections based on a common field. It works similar to a LEFT OUTER JOIN in SQL and returns matching documents from another collection as an array.

MongoDB also supports embedded documents, where related data is stored in a single document to avoid joins. Referenced documents are used when data is stored in separate collections and linked using a common field.

Points

- MongoDB uses $lookup for joins
- Works similar to LEFT OUTER JOIN
- Uses aggregation pipeline
- Combines multiple collections
- Based on common field values
- Supports embedding and referencing

8.OUTPUT:-

```
test> use ecommerceDB
switched to db ecommerceDB
ecommerceDB> show dbs
admin   40.00 KiB
config  12.00 KiB
local   40.00 KiB
ecommerceDB>
(To exit, press Ctrl+C again or Ctrl+D or type .exit)
ecommerceDB> db
ecommerceDB
ecommerceDB> db.createCollection("customers")
| db.createCollection("products")
| db.createCollection("orders")
| db.createCollection("payments")
| db.createCollection("cart")
{ ok: 1 }
ecommerceDB> show collections
cart
customers
orders
payments
products
ecommerceDB> db.customers.insertMany([
| {
| customer_id: 1,
| name: "Rahul Sharma",
| email: "rahul@gmail.com",
| city: "Pune"
| },
| {
| customer_id: 2,
| name: "Priya Patil",
| email: "priya@gmail.com",
| city: "Mumbai"
| },
| {
| customer_id: 3,
| name: "Amit Kumar",
| email: "amit@gmail.com",
| city: "Delhi"
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69d2aefd6821ac0c2e3682d1'),
    '1': ObjectId('69d2aefd6821ac0c2e3682d2'),
    '2': ObjectId('69d2aefd6821ac0c2e3682d3')
  }
}
```

```
ecommerceDB> db.products.insertMany([
| {
| product_id: 101,
| name: "Laptop",
| price: 55000,
| stock: 10
| },
| {
| product_id: 102,
| name: "Mobile",
| price: 25000,
| stock: 20
| },
| {
| product_id: 103,
| name: "Headphones",
| price: 2000,
| stock: 50
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69d2af156821ac0c2e3682d4'),
    '1': ObjectId('69d2af156821ac0c2e3682d5'),
    '2': ObjectId('69d2af156821ac0c2e3682d6')
  }
}
ecommerceDB> db.orders.insertMany([
| {
| order_id: 5001,
| customer_id: 1,
| product_id: 101,
| quantity: 1,
| total: 55000
| },
| {
| order_id: 5002,
| customer_id: 2,
| product_id: 102,
| quantity: 2,
| total: 50000
| },
| {
| order_id: 5003,
| customer_id: 3,
| product_id: 103,
| quantity: 3,
| total: 6000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69d2af226821ac0c2e3682d7'),
    '1': ObjectId('69d2af226821ac0c2e3682d8'),
    '2': ObjectId('69d2af226821ac0c2e3682d9')
  }
```

```
ecommerceDB> db.payments.insertMany([
| {
| payment_id: 9001,
| order_id: 5001,
| method: "UPI",
| status: "Paid"
| },
| {
| payment_id: 9002,
| order_id: 5002,
| method: "Card",
| status: "Paid"
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69d2af316821ac0c2e3682da'),
    '1': ObjectId('69d2af316821ac0c2e3682db')
  }
}
ecommerceDB> db.cart.insertMany([
| {
| cart_id: 1,
| customer_id: 1,
| product_id: 102,
| quantity: 1
| },
| {
| cart_id: 2,
| customer_id: 2,
| product_id: 103,
| quantity: 2
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69d2af3d6821ac0c2e3682dc'),
    '1': ObjectId('69d2af3d6821ac0c2e3682dd')
  }
}
ecommerceDB>
(To exit, press Ctrl+C again or Ctrl+D or type .exit)
ecommerceDB>
```

```
ecommerceDB> db.customers.find()
| db.products.find()
| db.orders.find()
| db.payments.find()
[
  {
    _id: ObjectId('69d2af316821ac0c2e3682da'),
    payment_id: 9001,
    order_id: 5001,
    method: 'UPI',
    status: 'Paid'
  },
  {
    _id: ObjectId('69d2af316821ac0c2e3682db'),
    payment_id: 9002,
    order_id: 5002,
    method: 'Card',
    status: 'Paid'
  }
]
ecommerceDB> db.customers.find({city: "Pune"})
[
  {
    _id: ObjectId('69d2aefd6821ac0c2e3682d1'),
    customer_id: 1,
    name: 'Rahul Sharma',
    email: 'rahul@gmail.com',
    city: 'Pune'
  }
]
ecommerceDB> db.products.find({price: {$gt: 20000}})
[
  {
    _id: ObjectId('69d2af156821ac0c2e3682d4'),
    product_id: 101,
    name: 'Laptop',
    price: 55000,
    stock: 10
  },
  {
    _id: ObjectId('69d2af156821ac0c2e3682d5'),
    product_id: 102,
    name: 'Mobile',
    price: 25000,
    stock: 20
  }
```

```
ecommerceDB> db.orders.aggregate([
| {
| $lookup: {
| from: "customers",
| localField: "customer_id",
| foreignField: "customer_id",
| as: "customer_details"
| }
| }
| ])
[
  {
    _id: ObjectId('69d2af226821ac0c2e3682d7'),
    order_id: 5001,
    customer_id: 1,
    product_id: 101,
    quantity: 1,
    total: 55000,
    customer_details: [
      {
        _id: ObjectId('69d2aefd6821ac0c2e3682d1'),
        customer_id: 1,
        name: 'Rahul Sharma',
        email: 'rahul@gmail.com',
        city: 'Pune'
      }
    ]
  },
  {
    _id: ObjectId('69d2af226821ac0c2e3682d8'),
    order_id: 5002,
    customer_id: 2,
    product_id: 102,
    quantity: 2,
    total: 50000,
    customer_details: [
      {
        _id: ObjectId('69d2aefd6821ac0c2e3682d2'),
        customer_id: 2,
        name: 'Priya Patil',
        email: 'priya@gmail.com',
        city: 'Mumbai'
      }
    ]
  },
  {
    _id: ObjectId('69d2af226821ac0c2e3682d9'),
    order_id: 5003,
    customer_id: 3,
    product_id: 103,
    quantity: 3,
    total: 6000,
    customer_details: [
      {
        _id: ObjectId('69d2aefd6821ac0c2e3682d3'),
        customer_id: 3,
        name: 'Amit Kumar',
        email: 'amit@gmail.com',
        city: 'Delhi'
      }
    ]
  }
]
ecommerceDB> db.products.updateOne(
| {product_id: 101},
| {$set: {price: 52000}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
ecommerceDB> db.products.updateOne(
| {product_id: 101},
| {$set: {price: 52000}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
ecommerceDB> db.customers.updateOne(
| {customer_id: 1},
| {$set: {city: "Mumbai"}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
ecommerceDB> db.customers.deleteOne({customer_id: 3})
{ acknowledged: true, deletedCount: 1 }
ecommerceDB> db.products.deleteOne({product_id: 103})
{ acknowledged: true, deletedCount: 1 }
ecommerceDB> |
```

9.CONCLUSION

In this assignment, NoSQL database concepts and MongoDB operations were studied and implemented for an E-Commerce and Order Management System. CRUD operations were performed to insert, retrieve, update, and delete data from collections such as customers, products, and orders. Various MongoDB features including operators, save method, sorting, and joins using $lookup were also implemented.

MongoDB provides a flexible schema, high scalability, and efficient data handling for large and dynamic applications. The use of NoSQL makes it easier to manage unstructured data and perform fast operations. Overall, MongoDB is suitable for modern applications like e-commerce systems due to its performance, flexibility, and scalability.